Space-efficient adversarially robust random-tape algorithms for missing item finding

Amit Chakrabarti, Manuel Stoeckl

doi:10.4230/LIPIcs.CCC.2024.28; CC BY-SA4.0

Editorial note. Original source: Amit Chakrabarti and Manuel Stoeckl, arXiv:2310.03634v2 (2024-07-03), https://arxiv.org/abs/2310.03634v2. License: Creative Commons Attribution-ShareAlike4.0, https://creativecommons.org/licenses/by-sa/4.0/.

Editorial note. Changes: selected complete author theorem/proof/context excerpts and original diagram/macros/bibliography in a standalone layout; omitted unrelated sections and author frontmatter. No mathematical corrections or additional proofs. This adapted presentation retains CC BY-SA4.0.

Allowed standard prerequisites (editorial).

- Previously known deterministic missing-item-finding fallback algorithm. Author lines1885-1892 explicitly cite Stoeckl23 for the simple ell-bit deterministic fallback at boundary parameters; full original bibliography retained.

Source reference. Original section 2.1: number-theory/sources/ccc-2024-28/native/main-content.tex, source lines 477–477. Optional high-level overview pointer; the delivered full algorithm, diagram and complete proof are self-contained without the introductory overview.

In this paper, we prove that for certain parameter regimes, MissingItemFinding (henceforth, MIF) is such a problem. In the problem MIF(n, ℓ), the input is a stream $\langle e_1, \ldots, e_\ell \rangle$ of ℓ integers, not necessarily distinct, with each $e_i \in \{1, \ldots, n\}$, where $1 \leq \ell \leq n$. The goal is as follows: having received the ith integer, output a number v in $\{1, \ldots, n\} \setminus \{e_1, \ldots, e_i\}$. We will be mostly interested in the setting $\ell = o(n)$, so the "trivial" upper bound on the space complexity of MIF(n, ℓ) is $O(\ell \log n)$, achieved by the deterministic algorithm that simply stores the input stream as is.

0.1 Groundwork for Our Results

To state our results about MIF, we need to introduce some key terminology. Notice that MIF is a *tracking problem*: an output is required after reading each input.[1] Thus, we view streaming algorithms as generalizations of finite state (Moore-type) machines. An algorithm $\mathcal{A}$ has a finite set of states Σ (leading to

[1] We do not consider algorithms with a "one-shot" guarantee, to only be correct at the end of the stream, because a) the adversarial setting requires tracking output b) for MIF and most other problems the difference in space complexity is generally small.

a space cost of $\log_2 |\Sigma|$), a finite input set $\mathcal{I}$, and a finite output set $\mathcal{O}$. It has a transition function $T\colon \Sigma \times \mathcal{I} \times \mathcal{R} \to \Sigma$ indicating the state to switch to after receiving an input, plus an output function $\gamma\colon \Sigma \times \mathcal{R} \to \mathcal{O}$ indicating the output produced upon reaching a state. How the final parameter (in $\mathcal{R}$) of T and γ is used depends on the type of randomness. We consider four cases, leading to four different models of streaming computation.

- *Deterministic.* The initial state of the algorithm is a fixed element of Σ, and T and γ are deterministic (they do not depend on the parameter in $\mathcal{R}$).
- *Random seed.* The initial state is drawn from a distribution $\mathcal{D}$ over Σ, and T and γ are deterministic. This models the situation that all random bits used count towards the algorithm's space cost.
- *Random tape.* The initial state is drawn from a distribution $\mathcal{D}$ over Σ.[2] The space $\mathcal{R}$ is a sample space; when the algorithm receives an input $e \in \mathcal{I}$ and is at state $\sigma \in \Sigma$, it chooses a random $\rho \in \mathcal{R}$ independent of all previous choices and moves to state $T(e, \sigma, \rho)$. However, γ is deterministic.[3] This models the situation that the algorithm can make random decisions at any time, but it cannot remember past random decisions without recording them (which would add to its space cost).
- *Random oracle.* The initial state is fixed; $\mathcal{R}$ is a sample space. A specific $R \in \mathcal{R}$ is drawn at the start of the algorithm and stays the same over its lifetime. When the algorithm is at state σ and receives input e, its next state is $T(e, \sigma, R)$. The output given at state σ is $\gamma(\sigma, R)$. This models the situation that random bits are essentially "free" to the algorithm; it can read from a long random string which doesn't count toward its space cost and which remains consistent over its lifetime. A random oracle algorithm can be interpreted as choosing a random deterministic algorithm, indexed by R, from some family.

These models form a rough hierarchy; they have been presented in (almost) increasing order of power. Every z-bit (2^z-state) deterministic algorithm can be implemented in any of the random models using z bits of space; the same holds for any z-bit random seed algorithm. Every z-bit random tape algorithm has a corresponding $(z + \log \ell)$-bit random oracle algorithm—the added space cost is because for a random oracle algorithm to emulate a random tape algorithm, it must have a way to get "fresh" randomness on each turn.[4]

Streaming algorithms are also classified by the kind of correctness guarantee they provide. Recall that we focus on "tracking" algorithms [BJWY20]; they present an output after reading each input item and this *entire sequence* of

[2] Requiring that this model use a fixed initial state could make some algorithms use one additional "INIT" state.

[3] Alternatively, we could associate a *distribution* of outputs to each state, or a function mapping (input, state) pairs to outputs. As these formulations are slightly more complicated to prove things with, and only affect the space usage of MISSINGITEMFINDING algorithms by an additive $O(\log n + \log \frac{1}{\delta})$ amount, we stick with the one state = one output convention.

[4] An alternative, which lets one express z-bit random tape algorithms using a z-bit random oracle variant, is to assume the random oracle algorithm has access to a clock or knows the position in the stream for free; both are reasonable assumptions in practice.

outputs must be correct. Here are three possible meanings of the statement "algorithm $\mathcal{A}$ is δ-error" (we assume that $\mathcal{A}$ handles streams of length ℓ with elements in $\mathcal{I}$ and has outputs in $\mathcal{O}$):

- *Static setting.* For all inputs $\tau \in \mathcal{I}^\ell$, running $\mathcal{A}$ on τ produces incorrect output with probability $\leq \delta$.
- *Adversarial setting.* For all (computationally unbounded) adaptive adversaries α (i.e., for all functions $\alpha\colon \mathcal{O}^\star \to \mathcal{I}$),[5] running $\mathcal{A}$ against α will produce incorrect output with probability $\leq \delta$.
- *Pseudo-deterministic setting.* There exists a canonical output function $f\colon \mathcal{I}^\star \to \mathcal{O}$ producing all correct outputs so that, for each $\tau \in \mathcal{I}^\ell$, $\mathcal{A}(\tau)$ fails to output $f(\tau)$ with probability $\leq \delta$.

Algorithms for the static setting are called "classic" streaming algorithms; ones for the adversarial setting are called "adversarially robust" streaming algorithms. All pseudo-deterministic algorithms are adversarially robust, and all adversarially robust algorithms are also classic.

As a consequence of Newman's theorem [New91], any random oracle or random tape algorithm in the static setting with error δ can be emulated using a random seed algorithm with only ε increase in error and an additional $O(\log \ell + \log\log |\mathcal{I}| + \log \frac{1}{\varepsilon\delta})$ bits of space. However, the resulting algorithm is non-constructive.

1 Preliminaries

Notation. Throughout this paper, $\log x = \log_2 x$, while $\ln x = \log_e x$. The set $\mathbb{N}$ consists of all positive integers; $[k] := \{1, 2, \ldots, k\}$; and $[a, b)$ is a half open interval of real numbers. For a condition or event E, the symbol $\mathbb{1}_E$ takes the value 1 if E occurs and 0 otherwise. The sequence (stream) obtained by concatenating sequences a and b, in that order, is denoted $a \circ b$. For a set S of elements in a totally ordered universe, $\textsc{sort}(S)$ denotes the sequence of elements of S in increasing order; $\binom{S}{k}$ is the set of k-element subsets of S; and $\textsc{seqs}(S, k) = \{\textsc{sort}(Y) : Y \in \binom{S}{k}\}$. We sometimes extend set-theoretic notation to vectors and sequences; e.g., for $y \in [n]^t$, write $y \subseteq S$ to mean that $\forall i \in [t] : y_i \in S$. For a set X, $\triangle[X]$ denotes the set of probability distributions over X, while $A \in_R X$ indicates that A is chosen uniformly at random from X. When naming probability distributions, we will use either calligraphic letters (e.g., $\mathcal{A}, \mathcal{D}$) or the letter μ; $A \sim \mu$ means that A is drawn from the distribution μ.

Lemma 1.1 (Multiplicative Azuma's inequality)**.** *Let $X_1, \ldots, X_t$ be $[0, 1]$ random variables, and $\alpha \geq 0$. If, for all $i \in [t]$, $\mathbb{E}[X_i \mid X_1, \ldots, X_{i-1}] \leq p_i$, then*

$$\Pr\left[\sum_{i=1}^{t} X_i \geq (1+\alpha)\sum_{i=1}^{t} p_i\right] \leq \exp\left(-((1+\alpha)\ln(1+\alpha) - \alpha)\sum_{i=1}^{t} p_i\right) \leq \exp\left(-\frac{\alpha^2}{2+\alpha}\sum_{i=1}^{t} p_i\right).$$

[5] By the minimax theorem, it suffices to consider deterministic adversaries.

On the other hand, if for all i, $\mathbb{E}[X_i \mid X_1, \ldots, X_{i-1}] \geq p_i$, then

$$\Pr\left[\sum_{i=1}^{t} X_i \leq (1-\alpha)\sum_{i=1}^{t} p_i\right] \leq \exp\left(-((1-\alpha)\ln(1-\alpha)+\alpha)\sum_{i=1}^{t} p_i\right) \leq \exp\left(-\frac{\alpha^2}{2}\sum_{i=1}^{t} p_i\right).$$

Proof of Theorem 1.1. The proof is modeled off that in [Sto23], which only addresses one direction. It is a straightforward blend of standard proofs of the Chernoff bound and of Azuma's inequality.

First, the $\geq$ direction. Choose, with foresight, $z = \ln(1+\alpha)$.

$$\begin{aligned}
&\Pr\left[\sum_{i=1}^{t} X_i \geq (1+\alpha)\sum_{i=1}^{t} p_i\right] \\
&= \Pr\left[\exp(z\sum_{i=1}^{t} X_i) \geq \exp(z(1+\alpha)\sum_{i=1}^{t} p_i)\right] \\
&\leq \frac{\mathbb{E}\exp(z\sum_{i=1}^{t} X_i)}{\exp(z(1+\alpha)\sum_{i=1}^{t} p_i)} \\
&\leq \frac{\mathbb{E}[e^{zX_1}\mathbb{E}[e^{zX_2}\ldots\mathbb{E}[e^{zX_t}|X_1,\ldots,X_{t-1}]\ldots|X_1]]}{\exp(z(1+\alpha)\sum_{i=1}^{t} p_i)}.
\end{aligned}$$

The innermost term $\mathbb{E}[e^{zX_t}|X_1,\ldots,X_{t-1}]$ is, by convexity of e^z, $\leq p_t e^z + (1-p_t) \leq e^{p_t(e^z-1)}$; after applying this upper bound, we can factor it out and bound the X_{t-1} term, and so on. Thus we continue the chain of inequalities to get:

$$\leq \frac{\exp\left((e^z-1)\sum_{i=1}^{t} p_i\right)}{\exp\left(z(1+\alpha)\sum_{i=1}^{t} p_i\right)} = \exp\left(-((1+\alpha)\ln(1+\alpha)-\alpha)\sum_{i=1}^{t} p_i\right).$$

For the other direction, set $z = \ln(1-\alpha)$, which is < 0. This time, $\mathbb{E}[e^{zX_t}|X_1,\ldots,X_{t-1}] \leq p_t e^z + (1-p_t)$ because p_t is a *lower bound* for $\mathbb{E}[X_t|X_1,\ldots,X_{t-1}]$, and z is negative. That $p_t e^z + (1-p_t) \leq e^{p_t(e^z-1)}$ still holds for negative z, so:

$$\begin{aligned}
\Pr\left[\sum_{i=1}^{t} X_i \leq (1-\alpha)\sum_{i=1}^{t} p_i\right] &= \Pr\left[\exp(z\sum_{i=1}^{t} X_i) \geq \exp(z(1-\alpha)\sum_{i=1}^{t} p_i)\right] \\
&\leq \ldots \leq \frac{\exp\left((e^z-1)\sum_{i=1}^{t} p_i\right)}{\exp\left(z(1-\alpha)\sum_{i=1}^{t} p_i\right)} \\
&= \exp\left(-((1-\alpha)\ln(1-\alpha)+\alpha)\sum_{i=1}^{t} p_i\right). \qquad \square
\end{aligned}$$

2 The Random Tape Upper Bound

In this section, we describe an adversarially robust random tape algorithm for $\textsc{mif}(n,\ell)$ which obtains error $\leq \delta$. See Original section 2.1 for a high level overview. The algorithm, shown in Listing 1, can be implemented for almost all pairs $\ell < n$, requiring only $\ell \leq n/64$ and $\ell \geq 4$ for its parameters to be

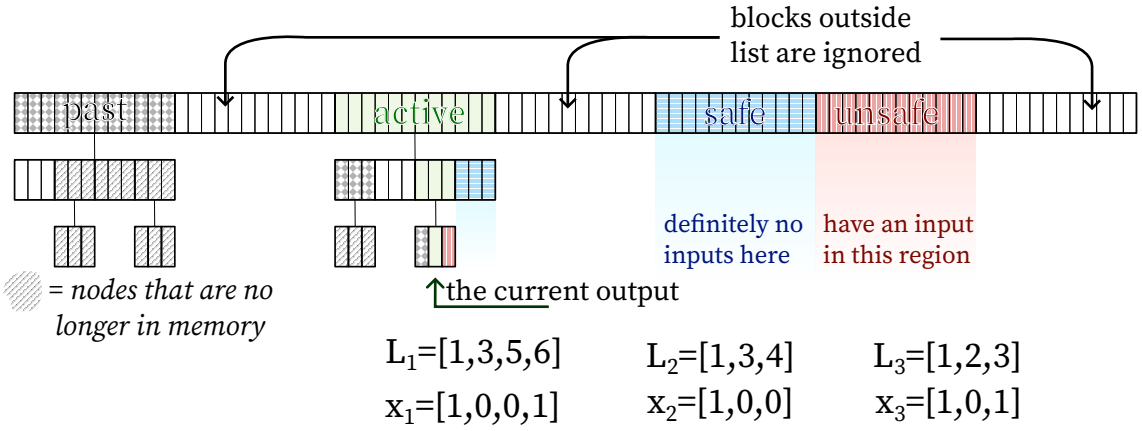


Figure 1: Diagram showing the state of the algorithm in Listing 1 and how it relates to the parts of the implicit random tree that the algorithm traverses. Positions on the horizontal axis correspond to different integers in $[n]$. To keep the example legible, we set parameters $d = 3$, $w_1 = 7, w_2 = 4, w_3 = 3$, and $b_1 = 4, b_2 = 3, b_3 = 3$.

meaningful. It can be seen as a multi-level generalization of the random oracle algorithm from [Sto23].

In order to prove that the algorithm in Listing 1 is correct, we will need some additional notation. Let $d, \alpha, b_1, \ldots, b_d, w_1, \ldots, w_d$ be as defined in Listing 1. It is helpful to view this algorithm as traversing over the leaves of a random tree of height d, in which:

- Every node v in the tree is associated with a subset S_v of $[n]$. We say a node is at level i if it is at depth $i - 1$. All nodes at a given level have disjoint associated subsets.
- The "root" ρ of the tree has S_ρ of size $\prod_{i=1}^{d} w_i$, and is at level 1
- Each node v at level i (depth $i - 1$) has b_{i+1} children; the set S_v is partitioned into w_{i+1} parts of equal size, and each child of v is associated with a random and unique one of these parts
- Each leaf node u, at depth d, is associated with a set of size 1, i.e., a single and unique integer in $[n]$. There are $\prod_{i=1}^{d} b_i$ leaf nodes in total.

See for example Figure 1. The algorithm maintains a view of just the branch of the tree from the root to the current leaf node. Its output will be the number associated to this leaf. For each node v on this branch, at level $i \in [d]$ (depth $i - 1$), it keeps a record of the positions $L_i \in [w_i]^{b_i}$ of its children, and a record $x_i \in \{0, 1\}^{b_i}$ indicating their status. There are four categories for child nodes:

- A node is PAST if the traversal over the tree passed through and leaf the node; past nodes are marked with a 1 in x_i.
- A node is ACTIVE if it is on the branch to the current leaf node; this is the node with the lowest index which is marked with a 0 in x_i.

- A node v is SAFE if it comes after the active node, and the adversary has never sent an input in S_v; safe nodes are marked with a 0 in x_i.
- A node v is UNSAFE if it comes after the active node, and the adversary did send an input in S_v; unsafe nodes are marked with a 1 in x_i.

The algorithm maintains these records as the adversary sends new inputs, marking safe child nodes v as unsafe if an element in S_v is received. The current leaf node is found by, from the root, following the chain of active nodes. If the adversary sends the value of the current leaf node, the algorithm will mark it by setting the corresponding entry in x_d to 1, thereby changing the value of the current active node. If every element of x_d is a now 1, this means that the adversary has sent an input for every child of the level d node on the current branch, so the algorithm marks the current active child of the level $d-1$ node with a 1, thereby moving the current branch to use a new level d node, u, which is *safe*.[6] It the "loads the positions of the children of u"—the tree being randomly generated, this is implemented by L_d being randomly sampled and x_d being reset to be all zeros—and proceeds.

While there are $\prod_{i=1}^{d} b_i$ leaf nodes in the ideal random tree, the algorithm's traversal of them may skip a fraction, because they (or one of their ancestors) were marked as unsafe. We say that such leaf nodes, along with those which were once active, have been KILLED.

Listing 1 uses the following lemma to set some of its parameters; the specific rounding scheme for the values $b_2, \ldots, b_{d-1}$ ensures that b_1 can decrease relatively smoothly as ℓ decreases. (Setting all $b_2 = \ldots = b_{d-1}$ to $\lfloor \alpha \rfloor$ can lead to having b_1 be significantly larger than necessary (by up to a factor $(3/2)^d = \ell^{O(1)}$); setting all $b_2 = \ldots = b_{d-1}$ to $\lceil \alpha \rceil$ would violate the $\prod_{i=1}^{d} w_i \le n$ constraint.)

Lemma 2.1. *Let $\alpha \ge 1$. Then for all $k \ge 0$, there exists an integer u depending on α and k so that*

$$\alpha^k \le \lceil \alpha \rceil^u \lfloor \alpha \rfloor^{k-u} \le 2\alpha^k .$$

Proof of Theorem 2.1. If α is an integer, we are done. Otherwise, with

$$u = \left\lceil \frac{k \log(\alpha/\lfloor \alpha \rfloor)}{\log(\lceil \alpha \rceil/\lfloor \alpha \rfloor)} \right\rceil, \quad \text{we have} \quad \lceil \alpha \rceil^u \lfloor \alpha \rfloor^{k-u} = \lfloor \alpha \rfloor^k (\lceil \alpha \rceil/\lfloor \alpha \rfloor)^u \ge \lfloor \alpha \rfloor^k (\lceil \alpha/\lfloor \alpha \rfloor \rceil)^k = \alpha^k .$$

Similarly, $\lceil \alpha \rceil^u \lfloor \alpha \rfloor^{k-u} \le \alpha^k \lceil \alpha \rceil/\lfloor \alpha \rfloor$, which is $\le 2\alpha^k$ because $\alpha \ge 1$ implies $\lceil \alpha \rceil/\lfloor \alpha \rfloor \le 2$. □

The following lemma is straightforward but tedious, and we defer its proof to Section 2.1.

[6] If the level $d-1$ node has no children marked with a 0 after this, we repeat the process at level $d-2$, and so on.

Listing 1 Adversarially robust random tape algorithm for $\textsc{mif}(n, \ell)$ with error $\leq \delta$

Requirements: $\ell \leq n/64$ and $\ell \geq 4$.

Parameters: $d = \min(\lceil \log \ell \rceil, \left\lfloor 2\frac{\log(n/4)}{\log 16\ell} \right\rfloor)$.

$$\alpha = \begin{cases} 2 & \text{if } \lceil \log \ell \rceil < \left\lfloor 2\frac{\log(n/4)}{\log 16\ell} \right\rfloor \\ \frac{(4\ell)^{2/(d-1)}}{(n/4)^{2/(d(d-1))}} & \text{otherwise} \end{cases}$$

Let u be chosen via Theorem 2.1, so that $\alpha^{d-2} \leq \prod_{i=2}^{d-1} b_i \leq 2\alpha^{d-2}$

$b_2 = \ldots = b_u = \lceil \alpha \rceil$, and $b_{u+1} = \ldots = b_{d-1} = \lfloor \alpha \rfloor$

$b_1 = \min(\ell + 1, \lceil 8\alpha \rceil + \lceil 3 \log 1/\delta \rceil)$; and $b_d = \left\lceil \frac{\ell}{\alpha^{d-1}} \right\rceil$

$w_1 = 16\ell$; and for each $i \in \{2, \ldots, d\}$, $w_i = \prod_{j=i}^{d} b_j$.

Let $\iota : [w_1] \times [w_2] \times \cdots \times [w_d] \to [n]$ be an arbitrary injective function

Initialization:

for $i \in [d]$ **do**
  $L_i \leftarrow$ random sequence without repetition in $[w_i]^{b_i}$
  $x_i \leftarrow (0, \ldots, 0) \in \{0, 1\}^{b_i}$

Update$(a \in [n])$:

if $a \notin \iota^{-1}[n]$ **then return** ▷ *Any integer not in $\iota^{-1}[n]$ can never be an output*
$v_1, \ldots, v_d = \iota^{-1}(a)$ ▷ *Map input into $[w_1] \times \ldots \times [w_d]$*
For $i \in [d]$, define $c_i = \min |\{j : x_i[j] = 0\}|$
if for all $i \in [d]$, $v_i = L_i[c_i]$ **then**
  ▷ *Move to the next leaf node, sampling new child node positions as necessary* ◁
  for $i = d, \ldots, 1$ **do**
    $x_i[c_i] \leftarrow 1$
    if x_i is the all-1s vector **then**
      if $i = 1$ **then abort** ▷ *If we reach $i = 1$, then even the root node is full*
      $L_i \leftarrow$ random sequence without repetition in $[w_i]^{b_i}$
      $x_i \leftarrow (0, \ldots, 0)$
    else break
else
  ▷ *Mark a branch as unsafe, if there was a hit* ◁
  Let j be the smallest integer in $[d]$ for which $v_j \neq L_j[c_j]$.
  if $\exists y \in [b_j]$ for which $L_j[y] = v_j$ **then**
    $x_j[y] \leftarrow 1$

Output $\to [n]$:

For $i \in [d]$, define $c_i = \min |\{j : x_i[j] = 0\}|$
return $\iota[(L_1[c_1], L_2[c_2], \ldots, L_d[c_d])]$

Lemma 2.2. *The parameters of Listing 1 satisfy the following conditions:*

$$\prod_{i=2}^{d} b_i \geq \frac{\ell}{\alpha}\,, \tag{1}$$

$$\prod_{i=2}^{d} b_i \leq \frac{4\ell}{\alpha}\,, \tag{2}$$

$$\prod_{i\in[d]} w_i \leq n\,. \tag{3}$$

We now prove the main lemma:

Lemma 2.3. *Listing 1 has error $\leq \delta$ in the adversarial setting.*

Proof of Theorem 2.3. We prove, using a charging scheme, that the probability of all leaf nodes in the random tree traversed by the algorithm being killed is $\leq \delta$.

The input of the adversary at any step falls into one of $d+2$ categories. For each $i \in [d]$, it could add an input which intersects the list of unrevealed child positions of the level i node, possibly killing $\prod_{j=i+1}^{d} b_i$ leaf nodes if it guesses correctly. It could also send the value of the current leaf node, thereby killing it (and only it). Finally, the adversary's input could be entirely wasted (outside $\iota([w_1] \times \ldots \times [w_d]$, repeating an input it made before, or in the region corresponding to one of the past nodes in the random tree); then no leaf nodes would be killed.

As the algorithm proceeds, for each node in the random tree (other than the root), we accumulate charge. When the algorithm's current branch changes to use new nodes, the charge on the old nodes is kept, and the new nodes start at charge 0.

When the adversary makes an input that is handled by level i ("query" at level i), for $i \in \{2,\ldots,d\}$, it *first* deposits one unit of charge at the active level i node. Then, if the query was a hit (i.e, ruled out some future subtree and made a child of a node in the current branch change from "safe" to "unsafe"), increase the number of killed nodes by the number of leaves for the subtree (namely, $\prod_{j=i+1}^{d} b_j$), and remove up to that amount of charge from the node. The definitions of $(w_j)_{j=2,\ldots,d}$ ensure that $\prod_{j=i+1}^{d} b_j = w_j/b_j$.

For the tth query, let K_t be the number of killed leaf nodes on the query, minus any accumulated charge on the node. Say the adversary picks a node at level i for $i \in \{2,\ldots,d-1\}$, and that node has $\hat{w}$ unexplored subtree regions (i.e, neither revealed because the algorithm produced outputs in them, nor because there was there a query at that subtree region in the past), and $\hat{b}$ gives the number of subtrees within this unexplored region. If $\hat{b}=0$, $\mathbb{E}[K_t|K_1,\ldots,K_{t-1}]=0$. Otherwise, let u be the number of subtrees which were revealed by the algorithm so far; we have $\hat{w} \leq w_j - u$ and $\hat{b} \leq b_j - u$. Then when we condition on the past increases in charge, the subtree regions within the unexplored region are still uniformly random; hence the probability of hitting a subtree is $\hat{b}/\hat{w}$. The number of leaf nodes killed by a hit is w_j/b_j. The total charge currently at the node must be $\geq (w_j - u - \hat{w}) - (\frac{w_j}{b_j})(b_j - u - \hat{b}) = \hat{b}\frac{w_j}{b_j} - \hat{w} + (\frac{w_j}{b_j} - 1)u \geq \hat{b}\frac{w_j}{b_j} - \hat{w}$, since each removed node consumes at most $\frac{w_j}{b_j}$ of the existing charge. Consequently,

the increase in killed leaf nodes if we hit is $\max(0, \frac{w_j}{b_j} - \max(0, \hat{b}\frac{w_j}{b_j} - \hat{w}))$, so the expected payoff is[7]:

$$\mathbb{E}[K_t \mid K_1, \ldots, K_{t-1}] = \frac{\hat{b}}{\hat{w}} \max\left(0,\, \frac{w_j}{b_j} - \max\left(0,\, \hat{b}\frac{w_j}{b_j} - \hat{w}\right)\right) \underset{\text{by Theorem 2.4}}{\leq} 1\,.$$

If the level is 1, then let $J \subseteq [16\ell]$ give the set of probed subtree positions, and H give the set of revealed subtree positions; since there are $\leq \ell$ queries, $|J|, |H|$ are both $\leq \ell$, and the probability of a query in an unexplored region to hit is $\leq \frac{b_1}{16\ell - |J \cup H|} \leq \frac{b_1}{14\ell}$. The charging scheme does not apply, so

$$\mathbb{E}[K_t \mid K_1, \ldots, K_{t-1}] \leq \frac{b_1}{14\ell} \cdot \prod_{j=2}^{d} b_j\,.$$

Thus in all cases, $\mathbb{E}[K_t \mid K_1, \ldots, K_{t-1}] \leq \max(1, \prod_{j=1}^{d} b_j/14\ell)$.

The total charge deposited on mid-level nodes is $\leq \ell$. The algorithm is guaranteed to succeed if the total number of leaves killed is less than the total number of leaves; i.e, if $\sum_{i\in[\ell]} K_t + \ell \leq \prod_{j=1}^{d} b_j$. Note that by Theorem 2.2 and the definition of b_1, $\prod_{j=1}^{d} b_j \geq b_1\ell/\alpha \geq 8\ell$. Consequently,

$$\ell + 7\mathbb{E}\left[\sum_{t=1}^{\ell} K_t\right] \leq 8\max\left(\ell,\, \frac{1}{14}\prod_{j=1}^{d} b_j\right) \leq 8\max\left(\frac{1}{8},\, \frac{1}{14}\right)\prod_{j=1}^{d} b_j \leq \prod_{j=1}^{d} b_j\,.$$

Now let $D_t = K_t / \prod_{j=2}^{d} b_j$, so that each $D_t \in [0,1]$. Writing events in terms of D_t lets us use Theorem 1.1 to bound the probability that too many leaves are killed:

$$\begin{aligned}
\Pr\left[\ell + \sum_{t\in[\ell]} K_t \geq \prod_{i=1}^{d} b_i\right] &\leq \Pr\left[\sum_{t\in[\ell]} K_t \geq 7\max\left(\ell, \frac{1}{14}\prod_{j=1}^{d} b_j\right)\right] \\
&\leq \Pr\left[\sum_{t\in[\ell]} D_t \geq 7\max\left(\ell/\prod_{j=2}^{d} b_j, \frac{b_1}{14}\right)\right] \\
&\leq \exp\left(-\frac{6^2}{2+6}\max\left(\ell/\prod_{j=2}^{d} b_j, \frac{b_1}{14}\right)\right) \\
&\leq \exp\left(-\frac{9b_1}{28}\right) \leq 2^{b_1 \cdot \frac{9}{28\ln 2}} \leq 2^{\lceil 3\log 1/\delta\rceil \frac{9}{28\ln 2}} \leq \delta\,. \square
\end{aligned}$$

In the preceding proof, we used the following:

Lemma 2.4. *Let $\hat{b}, \hat{w}, b, w$ be positive, and $\hat{b} \geq 1$. Then:*

$$\frac{\hat{b}}{\hat{w}}\left(\max\left(0,\, \frac{w}{b} - \max\left(\hat{b}\frac{w}{b} - \hat{w},\, 0\right)\right)\right) \leq 1\,.$$

[7]When the level is d and $b_j = w_j$, we in fact we have $K_t = 1$ always; but we do not need this stronger fact.

Proof of Theorem 2.4. If $\frac{\hat{b}}{\hat{w}} \le \frac{b}{w}$, then:

$$\frac{\hat{b}}{\hat{w}}\left(\max(0, \frac{w}{b} - \max(\hat{b}\frac{w}{b} - \hat{w}, 0))\right) \le \frac{\hat{b}}{\hat{w}}\frac{w}{b} \le 1\,.$$

Otherwise, $\frac{\hat{b}}{\hat{w}} \ge \frac{b}{w}$, and:

$$\begin{aligned}
&\frac{\hat{b}}{\hat{w}}\left(\max\left(0, \frac{w}{b} - \max\left(\hat{b}\frac{w}{b} - \hat{w}, 0\right)\right)\right) \\
&\quad\le \frac{\hat{b}}{\hat{w}}\left(\frac{w}{b} - \left(\hat{b}\frac{w}{b} - \hat{w}\right)\right) = \frac{\hat{b}}{\hat{w}}\left(\frac{w}{b} - \hat{b}\left(\frac{w}{b} - \frac{\hat{w}}{\hat{b}}\right)\right) \\
&\quad\le \frac{\hat{b}}{\hat{w}}\left(\frac{w}{b} - 1\left(\frac{w}{b} - \frac{\hat{w}}{\hat{b}}\right)\right) = \frac{\hat{b}}{\hat{w}}\frac{\hat{w}}{\hat{b}} = 1\,, \qquad \text{since } \hat{b} \ge 1 \text{ and } \frac{w}{b} - \frac{\hat{w}}{\hat{b}} \ge 0\ .\square
\end{aligned}$$

The proof of the following lemma is a mostly straightforward calculation, which we defer to Section 2.1.

Lemma 2.5. *Listing 1 uses*

$$O\left(\left\lceil \frac{(4\ell)^{2/(d-1)}}{(n/4)^{2/(d(d-1))}} \right\rceil (\log \ell)^2 + \min(\ell, \log 1/\delta)\log \ell\right) \tag{4}$$

bits of space, where $d = \min\left(\lceil \log \ell \rceil, \left\lfloor 2\frac{\log(n/4)}{\log(16\ell)} \right\rfloor\right)$. *A weaker upper bound on this is:*

$$O\left(\ell^{\frac{\log \ell}{\log n}} (\log \ell)^2 + \min(\ell, \log 1/\delta)\log \ell\right)\,.$$

If $\ell \ge 4$ and $\ell \le n/64$, then Theorem 2.5 and Theorem 2.3 together show that Listing 1 has error $\le \delta$ and space usage as bounded by eq. (4). To handle the cases where $\ell < 4$ and $\ell > n/64$, one can instead use the simple deterministic algorithm for $\textsc{mif}(n, \ell)$ from [Sto23], using only ℓ bits of space. As this is in fact less than the space upper bound from eq. (4), it follows that eq. (4) gives an upper bound on the space needed for a random tape, adversarially robust $\textsc{mif}(n, \ell)$ algorithm for any setting of parameters. Formally:

Theorem 2.6. *There is a family of adversarially robust random tape algorithms, where for* $\textsc{mif}(n, \ell)$ *the corresponding algorithm has* $\le \delta$ *error and uses*

$$O\left(\left\lceil \frac{(4\ell)^{\frac{2}{d-1}}}{(n/4)^{\frac{2}{d(d-1)}}} \right\rceil (\log \ell)^2 + \min(\ell, \log \tfrac{1}{\delta})\log \ell\right)$$

bits of space, where $d = \max\left(2, \min\left(\lceil \log \ell \rceil, \left\lfloor 2\frac{\log(n/4)}{\log(16\ell)} \right\rfloor\right)\right)$. *When* $\delta = 1/\mathrm{poly}(n)$ *a (weakened) space bound is* $O\left(\ell^{\log_n \ell}(\log \ell)^2 + \log \ell \log n\right)$.

2.1 Mechanical Proofs for Section 2

Proof of Theorem 2.2. First, we handle the case where $\lceil \log \ell \rceil < \left\lfloor 2\frac{\log(n/4)}{\log 16\ell} \right\rfloor$. Then $d = \lceil \log \ell \rceil \le \left\lfloor 2\frac{\log(n/4)}{\log 16\ell} \right\rfloor - 1$, and $\alpha = 2$. Note that $\frac{\ell}{2^{d-1}} \ge 1$ since

$2^{d-1} \le 2^{\lceil \ell \rceil - 1} \le 2\ell/2 = \ell$.

$$\prod_{i=2}^{d} b_i = \left\lceil \frac{\ell}{2^{d-1}} \right\rceil 2^{d-2} \ge \frac{\ell}{2} = \frac{\ell}{\alpha}$$
$$\prod_{i=2}^{d} b_i = \left\lceil \frac{\ell}{2^{d-1}} \right\rceil 2^{d-2} \le 2\frac{\ell}{2} \le \frac{4\ell}{\alpha}\,.$$

Since $b_d = \left\lceil \frac{\ell}{2^{d-1}} \right\rceil = \left\lceil \frac{2\ell}{2^{\lceil \log \ell \rceil}} \right\rceil \le 2$,

$$\begin{aligned}
\prod_{i \in [d]} w_i &= (16\ell) \prod_{i=2}^{d} \prod_{j=i}^{d} b_j \le (16\ell) \prod_{i=2}^{d} 2^{d-i+1} = (16\ell) 2^{d(d-1)/2} \\
&\le (16\ell)(2^{\lceil \log \ell \rceil})^{(d-1)/2} \le (16\ell)(2\ell)^{(d-1)/2} \\
&\le (16\ell)(2\ell)^{(\lfloor 2\frac{\log(n/4)}{\log 16\ell} \rfloor - 2)/2} \\
&\le (16\ell)(2\ell)^{\frac{\log(n/4)}{\log 16\ell} - 1} \\
&\le (16\ell)^{\frac{\log(n/4)}{\log 16\ell}} = \frac{n}{4} \le n\,.
\end{aligned}$$

Second, we consider the case where $d = \left\lfloor 2\frac{\log(n/4)}{\log(16\ell)} \right\rfloor$. Because $n \ge 64\ell$, $d \ge 2$, and so

$$d = \left\lfloor 2\frac{\log(n/4)}{\log 16\ell} \right\rfloor \ge \frac{2}{3} \cdot 2\frac{\log(n/4)}{\log 16\ell} = \frac{\log(n/4)}{\frac{3}{4}\log 16\ell} \ge \frac{\log(n/4)}{\log 4\ell}\,.$$

The second inequality used that $\frac{3}{4}(4+\log \ell) \le (2+\log \ell)$ for $\ell \ge 4$. Consequently,

$$\alpha = \left(\frac{(4\ell)^d}{n/4}\right)^{\frac{2}{d(d-1)}} \ge \left(\frac{(4\ell)^{\frac{\log(n/4)}{\log 4\ell}}}{n/4}\right)^{\frac{2}{d(d-1)}} = \left(\frac{n/4}{n/4}\right)^{\frac{2}{d(d-1)}} = 1\,.$$

We now prove Eq. 1. Because $\prod_{i=2}^{d-1} b_i \ge \alpha^{d-2}$,

$$\prod_{i=2}^{d} b_i = \left\lceil \frac{\ell}{\alpha^{d-1}} \right\rceil \prod_{i=2}^{d-1} b_i \ge \frac{\ell}{\alpha \prod_{i=2}^{d-1} b_i} \cdot \prod_{i=2}^{d-1} b_i = \frac{\ell}{\alpha}\,.$$

For Eq. 3, we observe that

$$d \le 2\frac{\log(n/4)}{\log(16\ell)} \qquad \Longrightarrow \qquad 16\ell \le (n/4)^{2/d} \qquad \Longrightarrow \qquad \ell \ge \alpha^{d-1} = \frac{(4\ell)^2}{(n/4)^{2/d}}\,,$$

and thus $\ell/\alpha^{d-1} \ge 1$, so $b_d = \lceil \ell/\alpha^{d-1} \rceil \le 2\ell/\alpha^{d-1}$. Then since $\prod_{i=2}^{d-1} b_i \le 2\alpha^{d-2}$,

$$\prod_{i=2}^{d} b_i \le \frac{2\ell}{\alpha^{d-1}} \prod_{i=2}^{d} b_i \le \frac{4\ell}{\alpha \prod_{i=2}^{d-1} b_i} \cdot \prod_{i=2}^{d-1} b_i \le \frac{4\ell}{\alpha}\,.$$

Finally, we prove Eq. 3. As noted above,

$$b_d \le \frac{2\ell}{\alpha^{d-1}} \le \frac{4\ell}{\alpha \prod_{i=2}^{d-1} b_j}\,.$$

Applying this fact to bound the left hand side of Eq. 3 gives:

$$\begin{aligned}
\prod_{i\in[d]} w_i &= (16\ell)\prod_{i=2}^{d}\prod_{j=i}^{d} b_j = 16\ell(b_d)^d \prod_{i=2}^{d-1}\prod_{j=i}^{d-1} b_j \\
&\le 16\ell\left(\frac{4\ell}{\alpha\prod_{i=2}^{d-1} b_j}\right)^{d-1}\prod_{i=2}^{d-1}\prod_{j=i}^{d-1} b_j \\
&\le \frac{4\cdot(4\ell)^d}{\alpha^{d-1}}\frac{1}{\prod_{i=2}^{d-1}\prod_{j=2}^{i-1} b_j} \\
&\le \frac{4\cdot(4\ell)^d}{\alpha^{d-1}}\frac{1}{\alpha^{(d-1)(d-2)/2}} \qquad \text{since } \prod_{j=2}^{d-1} b_j \ge \alpha^{d-2} \text{ and } b_2 \ge b_3 \ge \ldots \ge b_{d-1} \\
&= \frac{4\cdot(4\ell)^d}{\alpha^{d(d-1)/2}} = \frac{4\cdot(4\ell)^d}{\frac{(4\ell)^d}{n/4}} = n\,. \qquad \square
\end{aligned}$$

Proof of Theorem 2.5. Listing 1 only stores two types of data: for each $i \in [d]$, the vectors $L_i \in [w_i]^{b_i}$, and the vectors $x_i \in \{0,1\}^{b_i}$. These can be stored using $b_i \log w_i$ and b_i, bits respectively, for a total of:

$$\begin{aligned}
\sum_{i\in[d]} b_i \log(2w_i) &\le b_1\log(2w_1) + \sum_{i=2}^{d} b_i \log(2w_i) \\
&\le b_1\log(32\ell) + \sum_{i=2}^{d} b_i \log(2\prod_{j=i}^{d} b_i) \le \sum_{i=1}^{d} b_i \log(32\ell)\,,
\end{aligned}$$

since by Eq. 2, $\prod_{j=i}^{d} b_i \le 4\ell$.

We now observe that $b_d \le \lceil\alpha\rceil$. If $d = \lceil\log\ell\rceil$, then $\alpha = b_2 = \ldots = b_{d-1} = 2$ and $b_d = \lceil \ell/\alpha^{d-1}\rceil \le 2$. On the other hand, if $d = \left\lfloor 2\frac{\log(n/4)}{\log(16\ell)}\right\rfloor$, then $\alpha = \frac{(4\ell)^{2/(d-1)}}{(n/4)^{2/(d(d-1))}}$. We have:

$$(4\ell)^{d+1} = (4\ell)^{\left\lfloor 2\frac{\log(n/4)}{\log(16\ell)}\right\rfloor+1} \ge (4\ell)^{2\frac{\log(n/4)}{\log(16\ell)}} = (n/4)^2\,,$$

which implies

$$\alpha = \frac{(4\ell)^{2/(d-1)}}{(n/4)^{2/(d(d-1))}} \ge \frac{(4\ell)^{2/(d-1)}}{(4\ell)^{(d+1)/(d(d-1))}} = (4\ell)^{(2-\frac{d+1}{d})\cdot\frac{1}{d-1}} = (4\ell)^{\frac{1}{d}}\,.$$

Consequently,

$$b_d = \left\lceil \frac{\ell}{\alpha^{d-1}}\right\rceil = \left\lceil \frac{1}{4}\frac{4\ell}{\alpha^{d-1}}\right\rceil \le \left\lceil \frac{1}{4}(4\ell)^{1/d}\right\rceil \le (4\ell)^{1/d} \le \alpha\,.$$

With the bound on b_d, and the fact that $\alpha \ge 1$ in both cases, and that $d \le \lceil\log\ell\rceil$ we obtain:

$$\begin{aligned}
\sum_{i\in[d]} b_i &\le \min(\ell+1, \lceil 8\alpha\rceil + \lceil 3\log 1/\delta\rceil) + (d-1)\lceil\alpha\rceil \\
&\le \min(\ell, \lceil 3\log 1/\delta\rceil) + (7+d)2\alpha \\
&\le \min(\ell, \lceil 3\log 1/\delta\rceil) + 32\log\ell\left\lceil \frac{(4\ell)^{2/(d-1)}}{(n/4)^{2/(d(d-1))}}\right\rceil.
\end{aligned}$$

Multiplying this last quantity by $\log(32\ell)$ gives a space bound.

To obtain a much weaker, but somewhat more comprehensible upper bound on α, when $d = \left\lfloor 2\frac{\log(n/4)}{\log(16\ell)} \right\rfloor$, we note that:

$$\begin{aligned}\max_{\lambda\in\mathbb{N}\cap[2,\infty)} \log \frac{(4\ell)^{2/(\lambda-1)}}{(n/4)^{2/(\lambda(\lambda-1))}} &\le \log \max_{\lambda\in\mathbb{R}\cap[2,\infty)} \left(\frac{2}{\lambda-1}\log(4\ell) - \frac{2}{\lambda(\lambda-1)}\log(n/4)\right) \\ &\le \log\left(2\log(4\ell) \max_{\lambda\in\mathbb{R}\cap[2,\infty)} \left(\frac{1}{\lambda-1} - \frac{1}{\lambda(\lambda-1)}\frac{\log(n/4)}{\log(4\ell)}\right)\right).\end{aligned}$$

Let $\gamma = \frac{\log(n/4)}{\log(4\ell)}$; this is ≥ 1. Let $f(x) = \frac{1}{x-1}(1-\frac{\gamma}{x})$. We will now prove that $\max_{x\ge 2} f(x) \le \frac{1}{2\gamma}$. We note that when $x = 2$, we have:

$$f(2) = 1 - \frac{\gamma}{2} \le \frac{1}{2\gamma}.$$

Checking the other endpoint, we have:

$$\lim_{x\to\infty} \frac{1}{x-1}(1-\frac{\gamma}{x}) = 0.$$

Since $f(x)$ is differentiable on $[2,\infty)$, if it has a maximum other than at the endpoints, then it will occur when $\frac{d}{dx}f(x) = 0$. Solving this equation, we obtain:

$$\frac{d}{dx}f(x) = -\frac{1}{(x-1)^2} + \frac{\gamma(2x-1)}{(x(x-1))^2} = -\frac{1}{(x-1)^2}\left[1 - \frac{\gamma(2x-1)}{x^2}\right] = 0,$$

which is true iff $x^2 = \gamma(2x-1)$. The solutions to the quadratic equation are

$$x = \gamma - \sqrt{\gamma(\gamma-1)} \qquad \text{and} \qquad x = \gamma + \sqrt{\gamma(\gamma-1)}.$$

Since $\gamma \ge 1$, the $-$ branch has $x \le 1$, which is not in $[2,\infty)$. The $+$ branch is only in $[2,\infty)$ if $\gamma \ge \frac{4}{3}$. The value of $f(x)$ in this case is:

$$\begin{aligned}f(\gamma+\sqrt{\gamma(\gamma-1)}) &= \frac{1}{\gamma+\sqrt{\gamma(\gamma-1)}-1}\left(1 - \frac{\gamma}{\gamma+\sqrt{\gamma(\gamma-1)}}\right) \\ &= \frac{\sqrt{\gamma(\gamma-1)}}{2\gamma-1+2\sqrt{\gamma(\gamma-1)}} \\ &\le \frac{1}{4\sqrt{\gamma(\gamma-1)}} && \text{(since } \sqrt{\gamma(\gamma-1)} \le 2\gamma-1 \text{ for all } \gamma\ge 1\text{)} \\ &\le \frac{1}{2\gamma}. && \text{(since } \gamma \le 2\sqrt{\gamma(\gamma-1)} \text{ for all } \gamma \ge \frac{4}{3}\text{)}\end{aligned}$$

Thus, if $f(x)$ does have a maximum in $[2,\infty)$, it is $\le \frac{1}{2\gamma}$. We conclude that $f(x) \le \frac{1}{2\gamma}$ in all cases. This proves:

$$\log\alpha \le \max_{\lambda\in\mathbb{N}} \log\frac{(4\ell)^{2/(\lambda-1)}}{(n/4)^{2/(\lambda(\lambda-1))}} \le \log\left(2\log(4\ell)\frac{1}{2^{\frac{\log(n/4)}{\log(4\ell)}}}\right) \le \log\frac{\log(4\ell)^2}{\log(n/4)}. \square$$

References

[ABJ+22] Miklós Ajtai, Vladimir Braverman, T.S. Jayram, Sandeep Silwal, Alec Sun, David P. Woodruff, and Samson Zhou. The white-box adversarial data stream model. In *Proc. 41st ACM Symposium on Principles of Database Systems*, pages 15–27, 2022.

[ACGS23] Sepehr Assadi, Amit Chakrabarti, Prantar Ghosh, and Manuel Stoeckl. Coloring in graph streams via deterministic and adversarially robust algorithms. In *Proc. 42nd ACM Symposium on Principles of Database Systems*, pages 141–153, 2023.

[BEEO22] Omri Ben-Eliezer, Talya Eden, and Krzysztof Onak. Adversarially robust streaming via dense-sparse trade-offs. In *Symposium on Simplicity in Algorithms (SOSA)*, pages 214–227, 2022.

[BJWY20] Omri Ben-Eliezer, Rajesh Jayaram, David P. Woodruff, and Eylon Yogev. A framework for adversarially robust streaming algorithms. In *Proc. 39th ACM Symposium on Principles of Database Systems*, page 63–80, 2020.

[BKKS23] Vladimir Braverman, Robert Krauthgamer, Aditya Krishnan, and Shay Sapir. Lower bounds for pseudo-deterministic counting in a stream. *arXiv preprint arXiv:2303.16287*, 2023.

[BR93] Mihir Bellare and Phillip Rogaway. Random oracles are practical: A paradigm for designing efficient protocols. In *Proc. 1st ACM Conference on Computer and Communications Security*, pages 62–73, 1993.

[BY20] Omri Ben-Eliezer and Eylon Yogev. The adversarial robustness of sampling. In *Proc. 39th ACM Symposium on Principles of Database Systems*, pages 49–62. ACM, 2020.

[CGS22] Amit Chakrabarti, Prantar Ghosh, and Manuel Stoeckl. Adversarially robust coloring for graph streams. In *Proc. 13th Conference on Innovations in Theoretical Computer Science*, pages 37:1–37:23, 2022.

[Fei19] Uriel Feige. A randomized strategy in the mirror game. *arXiv preprint arXiv:1901.07809*, 2019.

[GGMW20] Shafi Goldwasser, Ofer Grossman, Sidhanth Mohanty, and David P. Woodruff. Pseudo-Deterministic Streaming. In *Proc. 20th Conference on Innovations in Theoretical Computer Science*, volume 151, pages 79:1–79:25, 2020.

[GGS23] Ofer Grossman, Meghal Gupta, and Mark Sellke. Tight space lower bound for pseudo-deterministic approximate counting. *arXiv preprint arXiv:2304.01438*, 2023.

[GS18] Sumegha Garg and Jon Schneider. The Space Complexity of Mirror Games. In *Proc. 10th Conference on Innovations in Theoretical Computer Science*, pages 36:1–36:14, 2018.

[HKM+20] Avinatan Hassidim, Haim Kaplan, Yishay Mansour, Yossi Matias, and Uri Stemmer. Adversarially robust streaming algorithms via differential privacy. In *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*, 2020.

[HW13] Moritz Hardt and David P. Woodruff. How robust are linear sketches to adaptive inputs? In *Proc. 45th Annual ACM Symposium on the Theory of Computing*, pages 121–130, 2013.

[Ind06] Piotr Indyk. Stable distributions, pseudorandom generators, embeddings, and data stream computation. *J. ACM*, 53(3):307–323, 2006.

[JP83] Kumar Joag-Dev and Frank Proschan. Negative association of random variables, with applications. *Ann. Stat.*, 11(1):286–295, 1983.

[JW23] Rajesh Jayaram and David P Woodruff. Towards optimal moment estimation in streaming and distributed models. *ACM Trans. Alg.*, 19(3):1–35, 2023.

[KMNS21] Haim Kaplan, Yishay Mansour, Kobbi Nissim, and Uri Stemmer. Separating adaptive streaming from oblivious streaming using the bounded storage model. In *Advances in Cryptology - CRYPTO 2021 - 41st Annual International Cryptology Conference, CRYPTO 2021, Virtual Event, August 16-20, 2021, Proceedings, Part III*, volume 12827 of *Lecture Notes in Computer Science*, pages 94–121. Springer, 2021.

[MN22a] Roey Magen and Moni Naor. Mirror games against an open book player. In *11th International Conference on Fun with Algorithms (FUN 2022)*, volume 226, pages 20:1–20:12, 2022.

[MN22b] Boaz Menuhin and Moni Naor. Keep that card in mind: Card guessing with limited memory. In *Proc. 13th Conference on Innovations in Theoretical Computer Science*, pages 107:1–107:28, 2022.

[New91] Ilan Newman. Private vs. common random bits in communication complexity. *Inform. Process. Lett.*, 39(2):67–71, 1991.

[Nis90] Noam Nisan. Pseudorandom generators for space-bounded computation. In *Proc. 22nd Annual ACM Symposium on the Theory of Computing*, pages 204–212, 1990.

[Nis93] Noam Nisan. On read once vs. multiple access to randomness in logspace. *Theoretical Computer Science*, 107(1):135–144, 1993.

[Sto23] Manuel Stoeckl. Streaming algorithms for the missing item finding problem. In *Proc. 34th Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 793–818, 2023. Full version at arXiv:2211.05170v1.

[WZ22] David P. Woodruff and Samson Zhou. Tight bounds for adversarially robust streams and sliding windows via difference estimators. In *Proc. 62nd Annual IEEE Symposium on Foundations of Computer Science*, pages 1183–1196, 2022.